

INSPECTAIR

Design Description

System- und Software-Architektur

Projekt: InspectAir – Luftqualitätsmessgerät
Version: 2.0
Datum: Februar 2026
Autoren: Maximilian Liebl, Cedric Stroh, Jannis Messer
Modul: Mikrocomputertechnik 1, DHBW Ravensburg
Dozent: Thomas Stingl (Airbus)

Dieses Dokument wurde unter Verwendung von KI-Tools (Claude, Anthropic) zur Überprüfung erstellt.

Inhaltsverzeichnis

1	Projektbeschreibung	2
1.1	Überblick	2
1.2	Kernfunktionen	2
2	Systemarchitektur	2
2.1	Hardware-Blockdiagramm	2
2.2	Bus-Zuordnung	3
3	Software-Architektur	3
3.1	Schichtenmodell	3
3.2	Dateistruktur	4
3.3	Zentrale Datenstruktur	4
3.4	Display-Architektur	4
3.5	Zustandsmaschine – Power Management	5
4	Bibliotheken und Abhängigkeiten	5
5	Kritische Designentscheidungen	6
6	Änderungshistorie	6

1 Projektbeschreibung

1.1 Überblick

InspectAir ist ein Luftqualitätsmonitor für Innenräume, basierend auf dem ESP32-S3. Das System erfasst kontinuierlich sechs Umweltparameter (CO₂, PM2.5, PM10, VOC, Temperatur, Luftfeuchtigkeit), erkennt Personen per 24 GHz-Radar und stellt alle Daten auf einem 4-Zoll-TFT-Display dar. Durch eine vierstufige Farbcodierung nach WHO-2021- und UBA-Grenzwerten können kritische Werte auf einen Blick erkannt werden.

1.2 Kernfunktionen

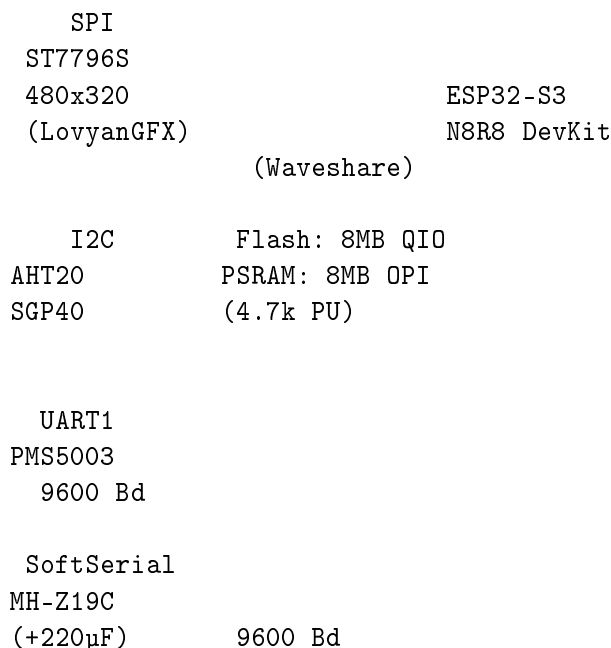
Das System bietet folgende Hauptfunktionen:

- Messung von CO₂, Feinstaub (PM1.0/PM2.5/PM10), VOC, Temperatur und Luftfeuchtigkeit
- Echtzeit-Anzeige auf 4-Zoll-Display (480×320 Pixel) über LVGL 9.2
- Fünf wählbare Bildschirmansichten (Overview, Detail, Analog, Bubble, Tree-Animation)
- Vierstufige Farbcodierung (Grün/Gelb/Orange/Rot) nach WHO/UBA-Grenzwerten
- Präsenzerkennung via 24 GHz FMCW-Radar für automatisches Display-Management
- Energiesparmodi: Display-Dimming, Display-Off und Light Sleep
- WiFi-Anbindung für NTP-Zeitsynchronisation und optionale Web-Fernsteuerung

2 Systemarchitektur

2.1 Hardware-Blockdiagramm

Das System besteht aus einem ESP32-S3 Mikrocontroller als zentraler Steuereinheit, verbunden mit fünf Sensoren über drei verschiedene Busse sowie einem TFT-Display über SPI:



UART2
LD2410C [TXS0108E]
(+220µF) 256000 Bd

2.2 Bus-Zuordnung

Bus	Sensor	Pins	Baudrate
SPI	ST7796S Display	MOSI:11, MISO:13, SCLK:12, CS:14, DC:9, RST:46	–
I2C	AHT20 + SGP40	SDA:8, SCL:18 (4,7 kΩ Pull-ups)	–
UART1	PMS5003	RX:16, TX:17	9600
SoftwareSerial	MH-Z19C	RX:4, TX:5	9600
UART2	LD2410C	RX:7, TX:6 (via Level Shifter)	256000
GPIO	UI-Button	GPIO1 (Active LOW, interner Pull-up)	–
GPIO	Backlight	GPIO3 (via LovyanGFX PWM)	–

3 Software-Architektur

3.1 Schichtenmodell

Die Software folgt einem 3-Schichten-Modell:

Schicht	Module
Application Layer	<code>main.cpp</code> – Hauptschleife, Timing, Button-Handler, Watchdog
Service Layer	<code>ui_manager</code> (5 Screens), <code>sensor_filter</code> (Moving Average), <code>sensor_history</code> (24h-Daten), <code>power_manager</code> (Energieverwaltung), <code>WifiClock</code> (NTP), <code>web_remote</code> (HTTP-Server)
Driver Layer	<code>lvgl_driver</code> (LVGL 9 + LovyanGFX), <code>aht_sgp</code> (I2C), <code>mhz19c</code> (Soft-Serial), <code>pms5003</code> (UART1), <code>ld2410c</code> (UART2)

3.2 Dateistruktur

Datei	Funktion
main.cpp	Hauptprogramm: setup(), loop(), Button-Handler, Sensor-Timing
pins.h	Zentrale GPIO-Pin-Definitionen
sensor_types.h	Datenstrukturen: <code>SensorReadings</code> , <code>AHT20_Data</code> , etc.
colors.h	RGB565-Farben, Grenzwerte, Klassifizierungsfunktionen
display_config.h	LovyanGFX-Konfiguration für ST7796S
app_config.h	Applikations-Konfiguration
lvgl_driver.cpp/h	LVGL 9 Display-Treiber, Flush-Callback, Brightness-Steuerung
ui_manager.cpp/h	Multi-Screen UI System (5 Screens), Update-Funktionen
aht_sgp.cpp/h	AHT20 + SGP40 I2C-Treiber
mhz19c.cpp/h	MH-Z19C CO ₂ -Treiber über SoftwareSerial
pms5003.cpp/h	PMS5003 Feinstaub-Treiber über UART1
ld2410c.cpp/h	LD2410C Radar-Treiber über UART2
sensor_filter.cpp/h	Moving-Average-Filter für Sensordaten
sensor_history.cpp/h	24-Stunden-Datenhistorie
power_manager.cpp/h	Display-Zustandsmaschine, Fade-Effekte, Light Sleep
WifiClock.cpp/h	WiFi-Verbindung und NTP-Zeitsynchronisation
web_remote.cpp/h	HTTP-Server für Fernsteuerung per Smartphone

3.3 Zentrale Datenstruktur

Alle Sensordaten werden in einer aggregierten Struktur zusammengefasst (definiert in `sensor_types.h`):

```
typedef struct {
    AHT20_Data aht;           // Temperatur/Feuchte
    SGP40_Data sgp;           // VOC-Index + Rohwert
    PMS5003_Data pms;         // PM1.0, PM2.5, PM10 (AE + SP)
    MHZ19C_Data mhz;          // CO2 in ppm + valid-Flag
    LD2410C_Data radar;       // Praesenz, Entfernung, Bewegung
    uint32_t timestamp;       // Zeitstempel (millis)
} SensorReadings;
```

3.4 Display-Architektur

Die Display-Ausgabe basiert auf einer zweistufigen Architektur:

Schicht 1 – LovyanGFX (`display_config.h`, `lvgl_driver.cpp`): Konfiguriert die SPI-Kommunikation mit dem ST7796S-Controller, verwaltet den Backlight-PWM über `setBrightness(0-255)` und stellt den Flush-Callback bereit. Dual-Buffer-Strategie mit PSRAM-Allokation (`LVGL_BUF_SIZE = 480*32`).

Schicht 2 – LVGL 9.2 (`ui_manager.cpp/h`): Verwaltet fünf verschiedene Bildschirme:

Enum	Screen	Beschreibung
UI_SCREEN_TREE	Baum-Animation	Startbildschirm mit animierter Baumgrafik
UI_SCREEN_OVERVIEW	Übersicht	Große AQI-Anzeige rechts, 2 Kacheln (Temp, Feuchte) links
UI_SCREEN_DETAIL	Detail	Kleine AQI, 4 Kacheln mit allen Messwerten
UI_SCREEN_ANALOG	Analog	Analoge Cockpit-Instrumente
UI_SCREEN_BUBBLE	Bubbles	Dynamische Kreise (Modern Bubbles Design)

Custom Fonts mit deutschen Umlauten sind in fünf Größen integriert: 12, 16, 20, 28 und 48 pt.

3.5 Zustandsmaschine – Power Management

Der `PowerManager` implementiert eine 4-Zustands-Maschine:

ACTIVE (45s ohne Präsenz) DIMMED (5 Min) OFF SLEEPING

(Präsenz erkannt → wakeUp)

Zustand	Beschreibung	Helligkeit
STATE_ACTIVE	Normalbetrieb, volle Displayhelligkeit	255 (100 %)
STATE_DIMMED	Nach 45 s ohne Präsenz, reduzierte Helligkeit	30 (~12 %)
STATE_OFF	Nach 5 Min ohne Präsenz, Display ausgeschaltet	0
STATE_SLEEPING	ESP32 Light Sleep, minimaler Stromverbrauch	0

Übergänge zwischen den Helligkeitsstufen erfolgen als Non-Blocking-Fade über `_startFade()` und `_updateFade()`.

4 Bibliotheken und Abhängigkeiten

Bibliothek	Version	Verwendung
LovyanGFX	1.1.16	Display-Treiber für ST7796S, Backlight-PWM
LVGL	9.2.2	UI-Framework, Widgets (Arc, Bar, Button, Image, Label, Flex)
Adafruit AHTX0	latest	AHT20 Temperatur-/Feuchte-Sensor
Adafruit SGP40	latest	SGP40 VOC-Sensor mit VOC-Index-Algorithmus
MH-Z19	1.5.4	MH-Z19C CO ₂ -Sensor-Treiber
PMS Library	1.1.0	PMS5003 Feinstaub-Sensor-Treiber
ld2410	0.1.3	LD2410C Radar-Sensor-Treiber
EspSoftwareSerial	8.2.0	Software-UART für MH-Z19C

5 Kritische Designentscheidungen

Entscheidung	Begründung
220 μ F Stützkondensatoren	Puffern Stromspitzen der 5V-Sensoren (MH-Z19C, LD2410C), die sonst Systemabstürze verursachen.
Level Shifter (TXS0108E)	LD2410C sendet 5V UART-Signale, die den 3,3V-ESP32 ohne Level Shifter zerstören würden.
SoftwareSerial für MH-Z19C	UART0 ist USB-Monitor, UART1 ist PMS5003, UART2 ist LD2410C – daher Software-UART (9600 Baud ist unkritisch).
LVGL 9.2 statt 8.3	Modernere API, besseres Speichermanagement, native Custom-Font-Unterstützung mit Umlauten.
PSRAM für LVGL-Buffer	Dual-Buffer in PSRAM entlastet den internen SRAM für Anwendungslogik.
Light Sleep statt Deep Sleep	Schnellere Aufwachzeit (~ 2 ms vs. ~ 300 ms), RAM bleibt erhalten, Radardaten persistent.
PETG 3D-Druck-Gehäuse	Temperaturbeständiger als PLA, bessere Schlagfestigkeit, gute Druckbarkeit mit Bambu Studio.

6 Änderungshistorie

Version	Datum	Autor	Änderung
1.0	Januar 2026	Team	Erstversion
2.0	Februar 2026	Team	LVGL 8.3→9.2, LovyanGFX 1.1.16, 5 Screens, SoftwareSerial, PETG-Gehäuse, aktualisierte Pin-Belegung, 4-Zustands-PowerManager, Web Remote.